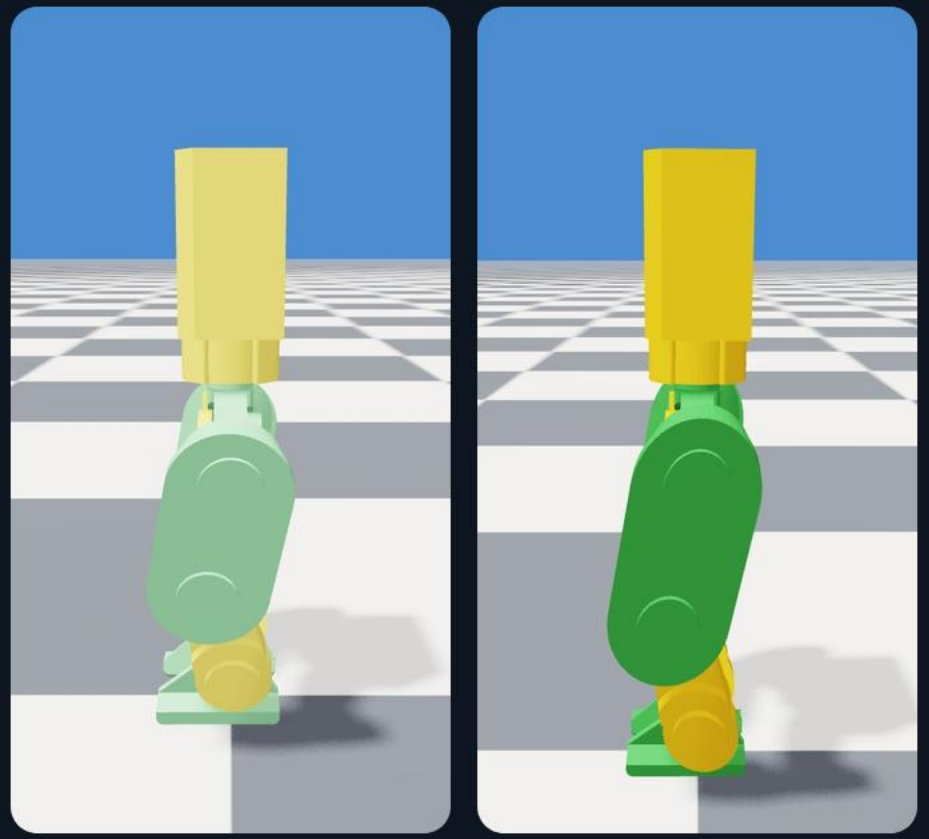
ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

Simulation for Humanoid

Week 6 — Customize humanoid robot

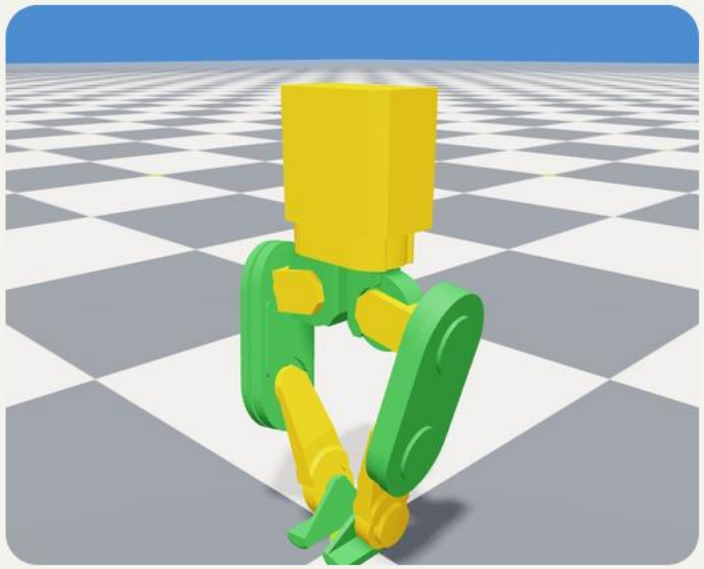
Isaac Lab 3.0 + Newton 2026 edition

Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai



WEEK 6

Agenda



01 Recap Week 4: poclegs project

02 Robot description: URDF, Xacro, USD

03 Lab 6.1: USD → URDF แล้วอ่านให้เป็น

04 Lab 6.2–6.3: ยืดขาแล้วเทรนใหม่

05 Exercises

RECAP FROM WEEK 4

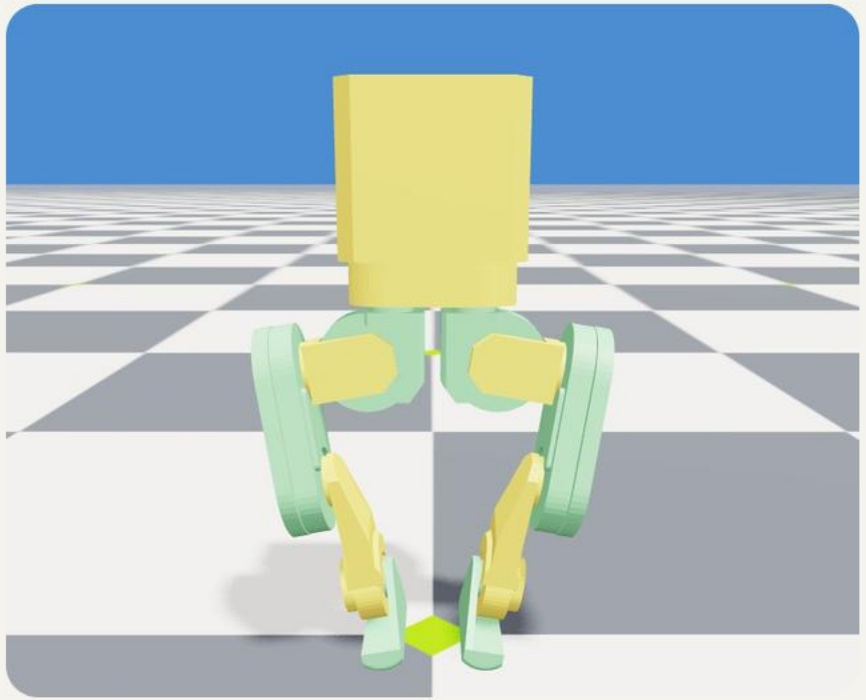
โปรเจกต์ poclegs_lab ของเรา

ROBOT = USD + ArticulationCfg

```
assets/poclegs.py
assets/data/poclegs/poclegs_newton.usd
```

TRAIN

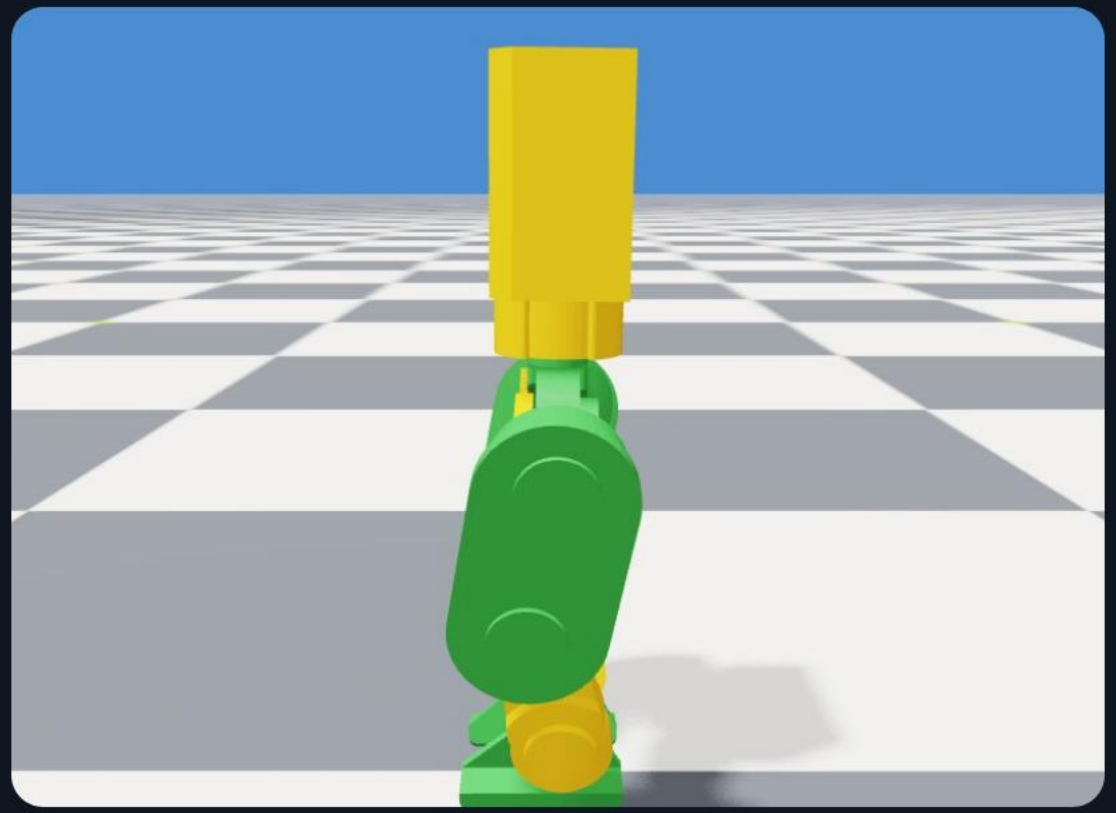
```
uv run isaaclab train --rl_library rsl_rl \
  --task Poclegs-Velocity-Flat
```



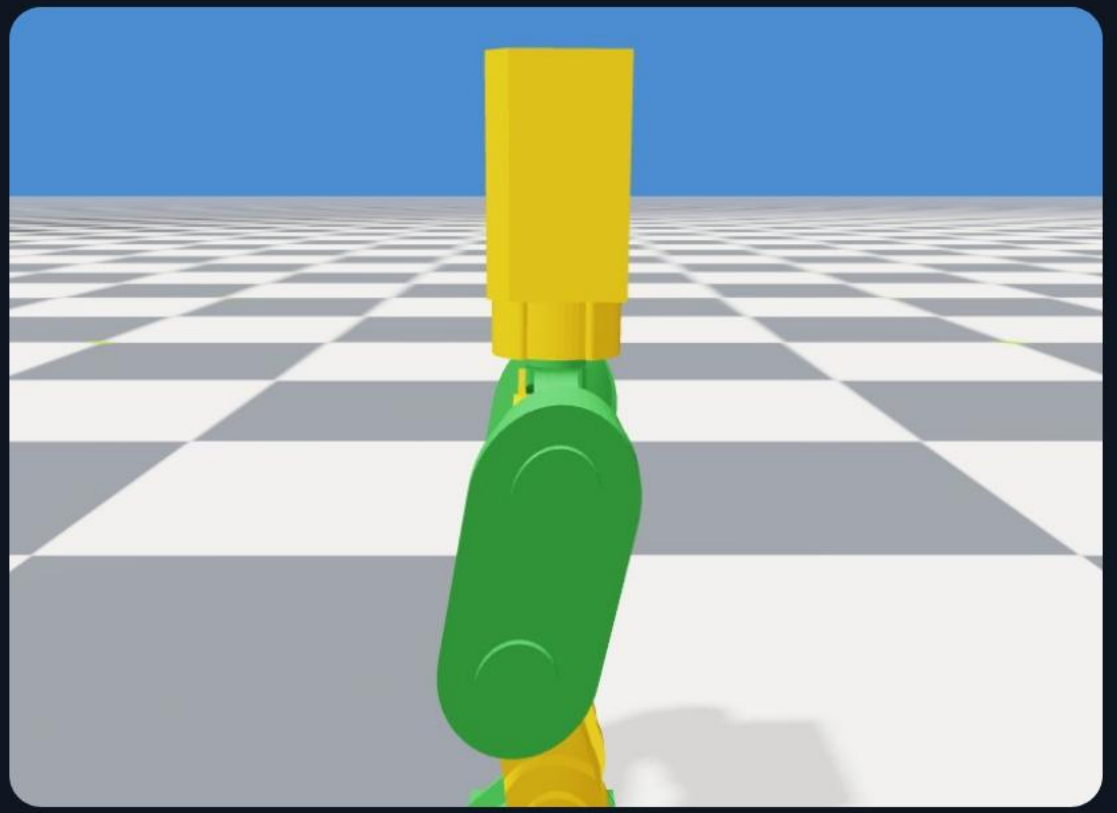
วันนี้เราจะแก้ “ร่างกาย” หุ่น แล้วดูว่าเดินต่างไปไหม

THIS WEEK

ขายาวขึ้น 1.3 เท่า แล้วยังเดินได้ไหม?



เดิม: ต้นขา 0.14 m, หน้าแข้ง 0.16 m, สูง 0.449 m

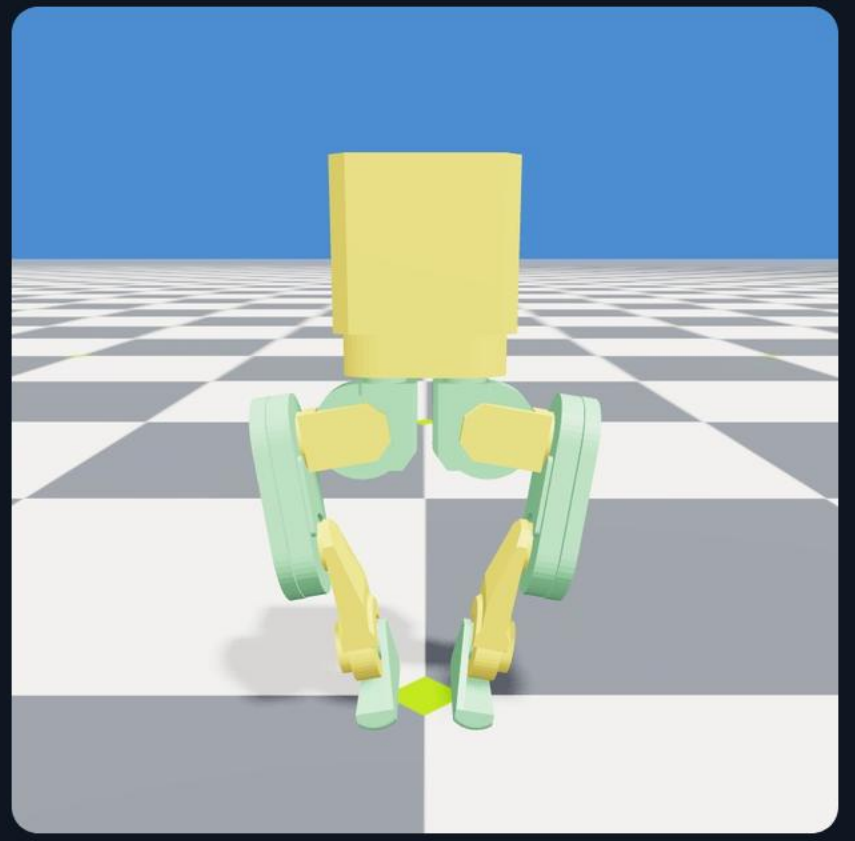


ใหม่: ต้นขา 0.182 m, หน้าแข้ง 0.208 m, สูง 0.539 m

LECTURE

Robot Description

simulator รู้ได้ยังไงว่าหุ่นมีกี่ขา ข้อพับตรงไหน หมุนรอบแกนอะไร?



WHY A ROBOT DESCRIPTION?

ต้องบอก simulator 3 เรื่อง

มีชิ้นส่วนอะไรบ้าง

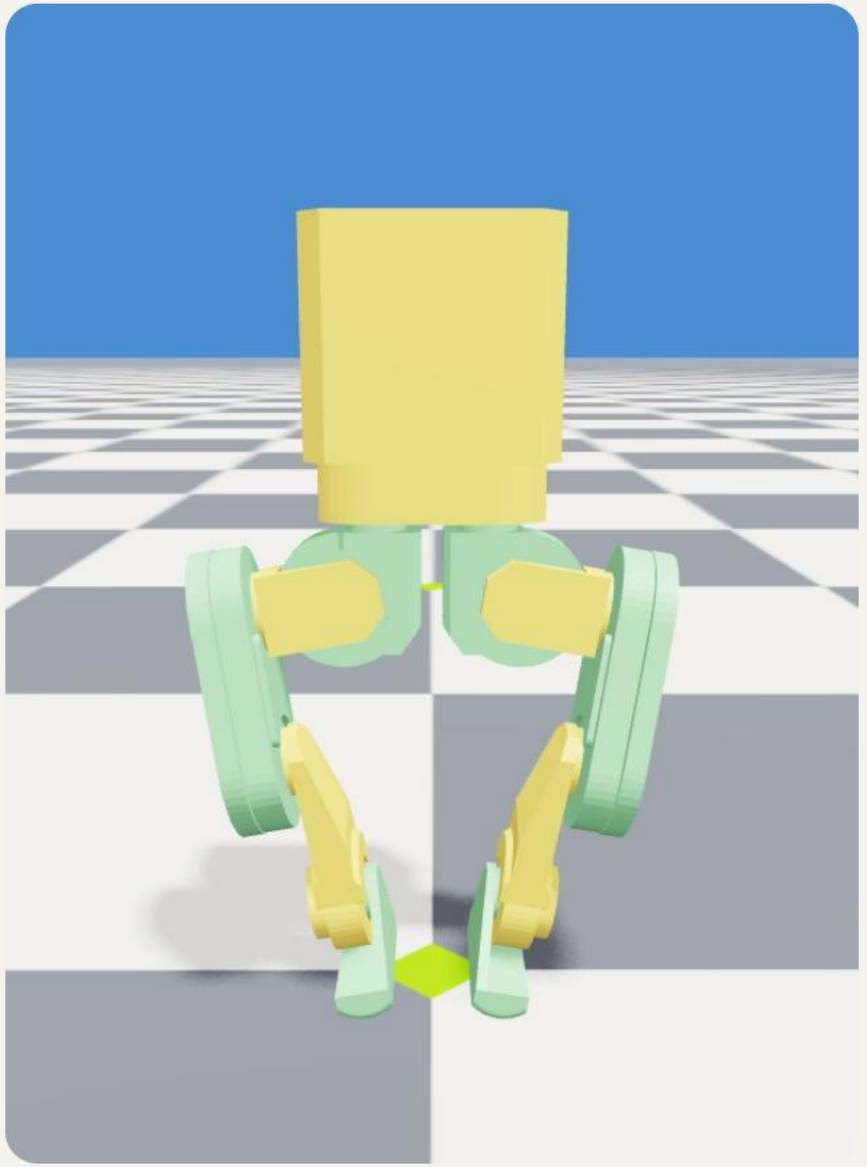
ลำตัว สะโพก ต้นขา หน้าแข้ง เท้า แต่ละชิ้นทรงไหน หนักเท่าไร

ต่อกันตรงไหน หมุนรอบแกนไหน

ตำแหน่งข้อต่อ แกนหมุน และมุมที่หมุนได้

ซ้ายกับขวาเป็นภาพสะท้อน

ขาขวามักสร้างจากขาซ้ายแล้วกลับด้าน วันนี้จะได้เห็นว่ามีผลยังไง



URDF

Unified Robot Description Format

ไฟล์ XML ที่บอกโครงสร้างหุ่น เป็นฟอร์แมตมาตรฐานของ ROS

link

ชิ้นส่วนแข็งหนึ่งชิ้น

joint

จุดหมุนที่ต่อ link สองชิ้น

visual

หน้าตาที่เรามองเห็น (mesh ละเอียด)

collision

รูปทรงไว้คิดการชน (ง่าย คำนวณเร็ว)

inertial

มวล จุดศูนย์ถ่วง โมเมนต์ความเฉื่อย

poclegs มี 11 link, 10 joint แต่ละ link มี visual/collision/inertial อยู่ข้างใน

WHERE ROBOT FILES COME FROM

ออกแบบใน CAD → ส่งออกเป็น mesh + URDF

Onshape
CAD บนเว็บ ใช้ onshape-to-robot export URDF

SolidWorks
sw_urdf_exporter

Fusion 360
add-in fusion2urdf

Blender
แต่ง mesh, ลด polygon



MESH · STL / OBJ / DAE
รูปทรงของแต่ละชิ้น (visual + collision)

URDF · XML
ชิ้นไหนต่อชิ้นไหน ด้วยข้อต่อแบบไหน มวลเท่าไร

poclegs ได้มาเป็น USD แล้ว วันนี้เลยต้องแปลงย้อนกลับเป็น URDF + OBJ

```xml
<robot name="poclegs">
  <link name="ll_kfe">
    <inertial> mass 1.0, inertia … </inertial>
    <visual> mesh ll_kfe_visual.obj </visual>
    <collision> mesh ll_kfe_collision.obj </collision>
  </link>

  <joint name="LL_FFE" type="revolute">
    <origin xyz="-0.042 -0.16 0" rpy="0 0 0"/>
    <parent link="ll_kfe"/>
    <child  link="ll_ffe"/>
    <axis xyz="1 0 0"/>
    <limit lower="-1.5708" upper="1.5708"
           effort="24" velocity="3.14159"/>
  </joint>
</robot>
```

URDF = XML

link กับ joint สลับกันเป็นต้นไม้

base → hr → haa → hfe → kfe → ffe
(ลำตัว → สะโพก × 3 → เข่า → ข้อเท้า)

หุ่นมี 10 joint, 11 link ขาซ้ายกับขวาเขียนซ้ำกันแทบทุกบรรทัด

โค้ดนี้ตัดมาจาก URDF ของ poclegs ที่เราจะ export วันนี้

XACRO = XML + MACRO

ขาซ้ายกับขวาเหมือนกัน ไม่ต้องเขียน 2 รอบ

เขียน macro ครั้งเดียว

```
<xacro:macro name="leg" params="side x">
  <link name="${side}_thigh"/>
  <link name="${side}_shank"/>
  <joint name="${side}_knee" type="revolute">
    <origin xyz="${x} -0.14 0"/> …
  </joint>
</xacro:macro>
```

เรียกใช้ 2 ครั้ง

```
<xacro:leg side="left"  x="0.05"/>
<xacro:leg side="right" x="-0.05"/>
```

reuse โค้ดได้ แล้ว **parametrize** ความยาวขาเป็นตัวแปร

ก่อนเอาไปใช้ ต้อง compile เป็น URDF

```
xacro robot.xacro > robot.urdf
```

THE PIPELINE

จาก template ไปถึง simulation

Xacro
template · macro

→

URDF
robot description (XML)

→

USD
simulation asset

→

Isaac Lab
Newton / PhysX

```
xacro robot.xacro > robot.urdf
```

```
urdf_to_usd.py · UrdfConverter
```

```
UsdFileCfg(usd_path=…)
```

วันนี้: แปลง USD เป็น URDF แก้ความยาวขา แล้วแปลงกลับเป็น USD เพื่อเทรน

WHY NOT URDF DIRECTLY?

URDF บอกโครงหุ่นได้ แต่ไม่พอสำหรับ sim

เรื่อง	URDF	USD
ฟิสิกส์	มวล, ขีดจำกัด, damping/friction ของข้อต่อ	drive, collision group, contact, solver
หน้าตา	สี + texture แบบง่าย	material แบบเต็ม (MDL / shader), แสง
sensor	มาตรฐานไม่มี (ต้องใช้ extension ของ Gazebo)	กล้อง, contact, IMU
ฉาก	หุ่นตัวเดียว	ทั้งฉาก: หุ่นหลายตัว + พื้น + วัตถุ
แยกไฟล์ / ซ้อนชั้น	ไฟล์เดียว (include ผ่าน xacro)	layer, reference, variant

วันนี้จะเจอเอง: แปลง USD เป็น URDF แล้ว collision group หาย ต้องปิด self-collision เอง

USD

Universal Scene Description

Pixar Animation Studios สร้างไว้ทำหนัง แล้วกลายมาเป็นมาตรฐานของ simulation

Pixar / OpenUSD

ใช้เก็บฉากหนังทั้งเรื่อง เป็น open source

NVIDIA Omniverse

Isaac Sim ทำงานบน USD

Newton

อ่าน UsdPhysics schema ได้เอง ไม่ต้องมี Isaac Sim

```
.usd / .usdc = binary
```

```
.usda = text อ่านได้
```

```
from pxr import Usd
```

INSIDE POCLEGS.USD

USD ซ้อนเป็นชั้น ๆ

```
poclegs.usd           # 2 KB · แค่รวมชั้น
└ configuration/
  ├ poclegs_base.usd     # 510 KB · mesh
  ├ poclegs_physics.usd  # 5 KB · ฟิสิกส์
  └ poclegs_sensor.usd   # 1 KB
```

แก้ฟิสิกส์ได้โดยไม่แตะ mesh ส่วน make_newton_usd.py รวมทุกชั้นเป็นไฟล์เดียว (flatten)

geometry
mesh ของ 11 ชิ้น (visual + collision)

physics
RigidBody, Mass, RevoluteJoint, Drive, ArticulationRoot

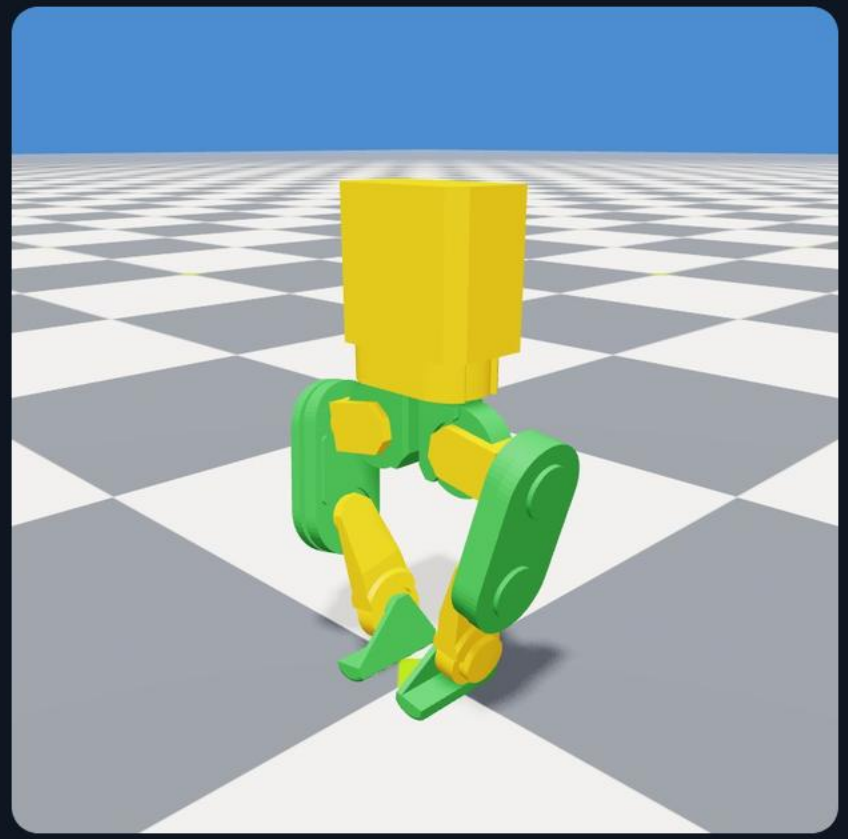
material
สีเหลือง/เขียวของชิ้นส่วน

collision group
กันหุ่นชนตัวเอง แต่ URDF เก็บค่านี้ไม่ได้

HANDS-ON · LAB 6.1

USD → URDF

poclegs มีแค่ไฟล์ USD ต้อง export เป็น URDF ก่อน ถึงจะแก้ได้ง่าย



STEP 0: เตรียมโปรเจกต์ WEEK 4 ให้ทำ WEEK 6 ได้

ก๊อปสคริปต์ 4 ตัว + เพิ่ม importers ใน pyproject.toml

ลบ/แก้ เพิ่ม/ค่าใหม่

ก่อน โปรเจกต์จาก Week 4

```
# pyproject.toml
dependencies = [
    "isaaclab-dev[rsl-rl]",
]

scripts/
├── list_envs.py
└── eval_policy.py   # จาก Week 4 Step 4

env_cfg.py: Rough · Flat (· Tuned ถ้าทำ Ex 4.3)

$ uv run python scripts/urdf_to_usd.py ...
→ ยังรันไม่ได้ (ไม่มีตัวแปลง URDF ในโปรเจกต์)
```

หลัง $SUP = Week06_Customize_Robot/week6-supplementary-file

```
# pyproject.toml
dependencies = [
    "isaaclab-dev[rsl-rl]",
    "isaaclab-dev[importers]",     # เพิ่ม แล้ว uv sync
]
scripts/      # cp $SUP/scripts/*.py scripts/
├── usd_to_urdf.py     # Lab 6.1
├── check_urdf.py      # Lab 6.1
├── scale_leg.py       # Lab 6.2 · Exercise
└── urdf_to_usd.py     # Lab 6.2

env_cfg.py: + terms/tuned.py (ถ้ายังไม่มี -Flat-Urdf)

$SUP/expected_output/*.urdf  # ไว้ diff ตรวจงาน
$SUP/pretrained/urdf1x, long13  # ตัวเทียบ Exercise
```

ถ้าใช้ฉบับเสร็จ Week04_RL_poclegs/poclegs_lab ข้ามหน้านี้ได้เลย ลองทำตามนี้กับโปรเจกต์ที่สร้างจาก template แล้ว ได้ URDF ตรงกับ expected_output ทุกไฟล์

TWO WAYS

แปลงไฟล์ได้ 2 ทาง

1 · Isaac Sim GUI (เริ่มตรงนี้)

USD → URDF: **File › Export to URDF**

URDF → USD: **File › Import** แล้วเลือกไฟล์ .urdf

กดเมนูอย่างเดียว เห็นผลทันที แต่ต้องเปิด Isaac Sim ทั้งตัว (หน้าถัดไป)

2 · สคริปต์ (สาย Newton)

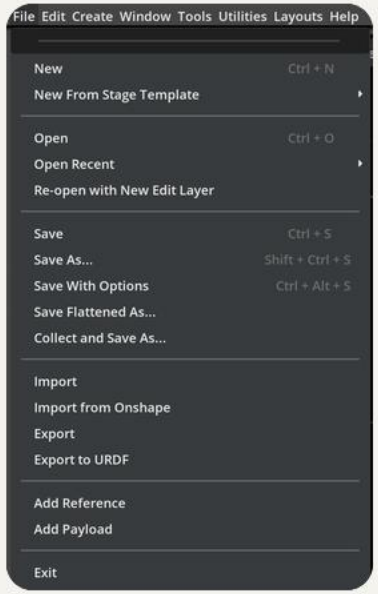
อ่าน USD ด้วย pxr แล้วเขียน URDF + mesh OBJ ขากลับใช้ตัวแปลงของ Isaac Lab ไม่ต้องเปิด Isaac Sim

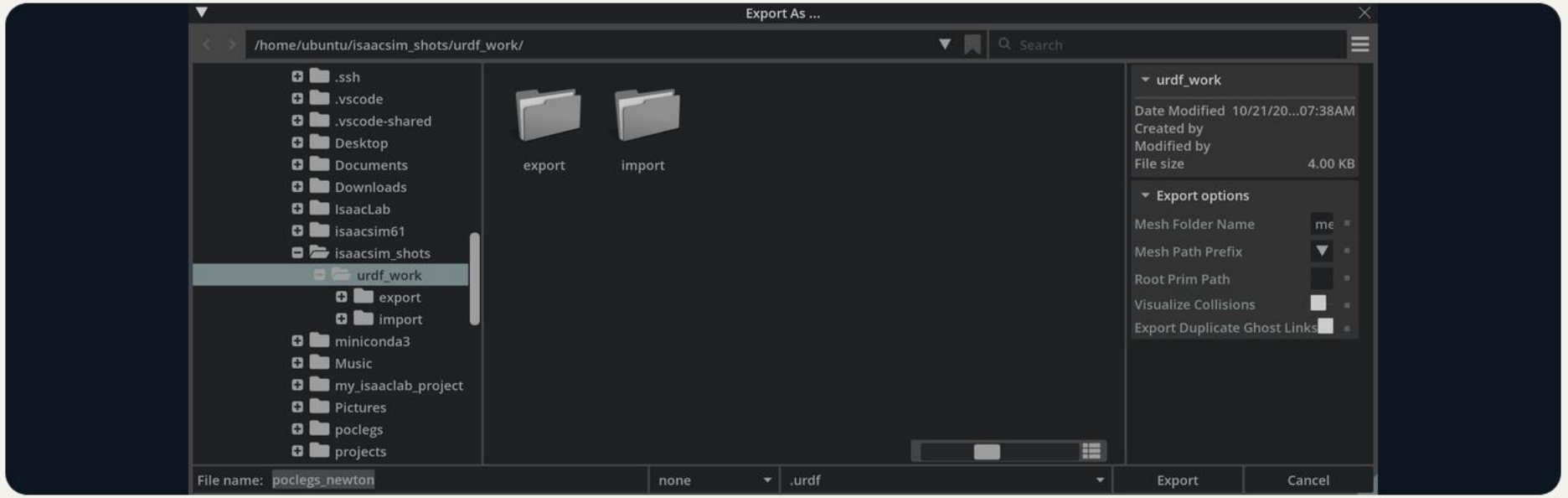
```
scripts/usd_to_urdf.py
scripts/urdf_to_usd.py
scripts/check_urdf.py
```

ใช้ใน Lab 6.1–6.3: รันซ้ำได้ ได้ไฟล์ตรงกับ expected_output

ทางที่ 1 · ISAAC SIM GUI

USD → URDF: File › Export to URDF





1. File › Open `poclegs_newton.usd` **2.** File › Export to URDF **3.** เลือกโฟลเดอร์ ตั้งชื่อ แล้วกด **Export** → ได้ `poclegs.urdf` + `meshes/`

ข้อต่อตรงกับสคริปต์: ลองสุ่มท่า 20 ท่า ตำแหน่งทุก link ต่างกันไม่ถึง 0.002 mm

แต่ไฟล์หน้าตาต่าง: มี joint `floating_base` และขาขวาใช้ rpy 180° แทน → ใน Lab ใช้ไฟล์จากสคริปต์

ASSIGNMENT 6.1 · EXPORT

รัน usd_to_urdf.py ได้ URDF + mesh 22 ไฟล์

```
$ cd ~/projects/poclegs_lab  # หรือฉบับเสร็จ Week04_RL_poclegs/poclegs_lab
$ D=src/poclegs_lab/assets/data
$ uv run python scripts/usd_to_urdf.py $D/poclegs/poclegs_newton.usd $D/urdf
wrote …/urdf/poclegs.urdf
```

ดึงอะไรจาก USD บ้าง

RigidBodyAPI → link, RevoluteJoint → joint
localPos/localRot → origin, axis, limit (องศา → rad)
Mesh → OBJ, สีจาก material

ที่ต้องคำนวณเพิ่ม

USD ของ poclegs ตั้ง inertia = 0 ให้ engine คิดเอง แต่ URDF ต้องมีค่า เลยคิดจาก convex hull ของ collision mesh × มวลจริง

ANATOMY · JOINT

joint หนึ่งตัวมี 5 ส่วน

```
<joint name="LL_KFE" type="revolute">  ①
  <origin xyz="-0.0025 -0.14 0" rpy="0 0 0"/>  ②
  <parent link="ll_hfe"/> <child link="ll_kfe"/>  ③
  <axis xyz="1 0 0"/>  ④
  <limit lower="-2.356" upper="2.356" effort="30" velocity="1.571"/>  ⑤
```

① ชื่อ + ชนิด
revolute = หมุนได้ในช่วงจำกัด

② origin
joint อยู่ห่างจาก joint ก่อนหน้า 0.14 m = **ความยาวต้นขา**

③ parent → child
ต้นขา → หน้าแข้ง

④ axis
"1 0 0" = หมุนรอบแกน x ของ link

⑤ limit
มุม (rad), แรง (Nm), ความเร็ว (rad/s)

ระวัง
velocity 1.571 rad/s เป็นเพดานจริงของ solver บน Newton ส่วน task Tuned / Urdf ปลดด้วย joint_velocity_limit แล้ว (Week 4)

ANATOMY · LINK

link หนึ่งชิ้นมี 3 ส่วน

inertial

```
<mass value="1.0"/>
<origin xyz="-0.017
  -0.109 0"/>
<inertia ixx=… />
```

มวล จุดศูนย์ถ่วง ความเฉื่อย ถ้าผิด หุ่นจะล้มง่ายและเทรนไม่นิ่ง

visual

```
<origin xyz="-0.158
  0.2125 0"/>
<mesh … scale="1 1 1"/>
```

mesh ที่มองเห็น ใช้ origin เลื่อนจาก frame ของ link (mesh วาดในพิกัด CAD)

collision

```
<mesh filename=
  "ll_kfe_collision.obj"/>
```

รูปทรงที่ใช้คิดการชน ตัวแปลงทำเป็น convex hull ให้คำนวณเร็ว

จะยืดขา ต้องแก้ทั้ง joint origin (ตำแหน่งข้อถัดไป) และ visual/collision (ให้ mesh ยาวตาม)

FRAMES

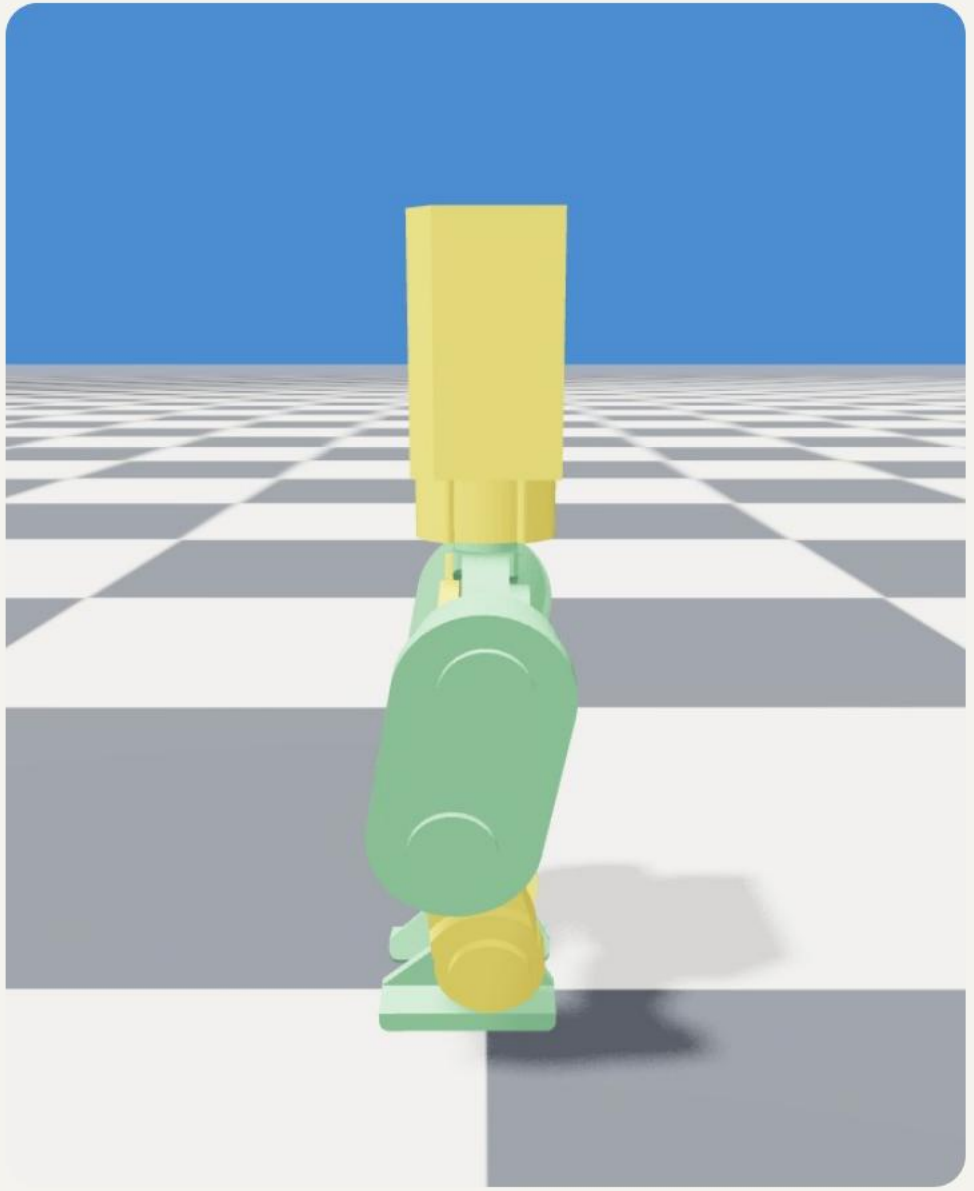
ทำไมขาชี้ไปทาง -y ไม่ใช่ -z?

1. CAD ต้นฉบับใช้ **y ชี้ขึ้น** แต่ Isaac / Newton ใช้ **z ชี้ขึ้น**

2. joint สะโพก LL_HR เลยหมุน frame +90° รอบแกน x:
   `rpy="1.5708 0 0"`

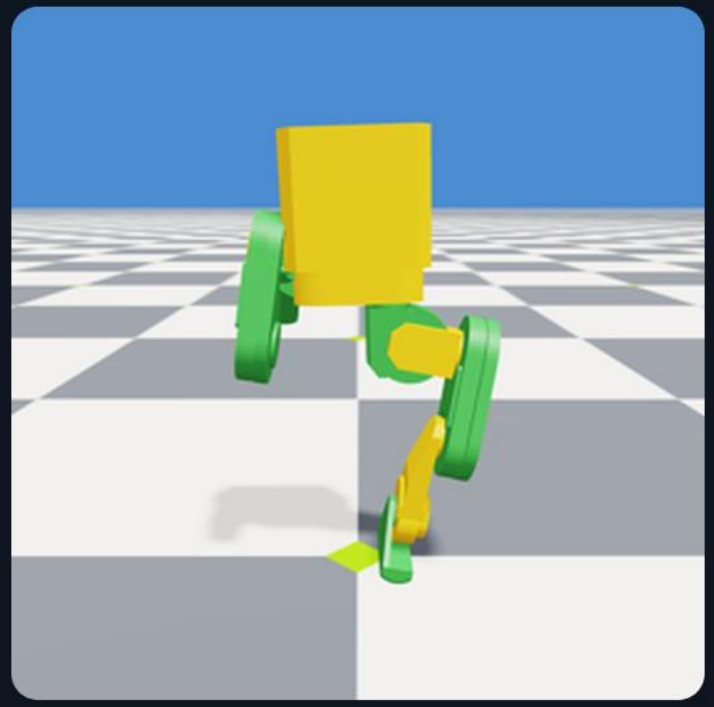
3. ทุก link ใต้ข้อนี้ **–y คือทิศลงพื้น** ความยาวขาเลยอยู่ในค่า y ของ origin

ขาสองข้างห่างกันตามแกน x (±0.05 m) แปลว่าด้าน “หน้า” ของหุ่นคือแกน y ของลำตัว

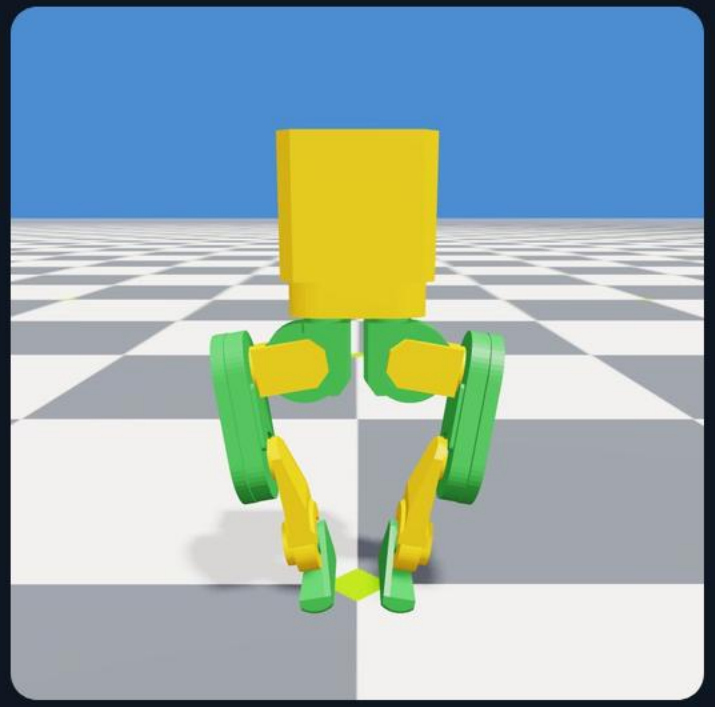


PITFALL: เจอเองตอนเตรียม WORKSHOP

ขาซ้ายปกติ แต่ขาขวาพลิก 180°



ก่อนแก้



หลังแก้

USD เก็บ joint frame ไว้ทั้งสองฝั่ง: `localRot0` (ฝั่ง parent) และ `localRot1` (ฝั่ง child)

ขาซ้าย localRot1 = 0° แต่ **LR_HR มี localRot1 = 180°** เพราะขาขวาสร้างจากขาซ้ายแล้วกลับด้าน

$$\text{origin} = J0 \cdot J1^{-1}$$
$$\text{axis} = R1 \cdot \text{axis}$$

VERIFY, DON’T TRUST

ตรวจ URDF ด้วย forward kinematics

```
$ uv run python scripts/check_urdf.py $D/poclegs/poclegs_newton.usd $D/urdf/poclegs.urdf
LL_HR    pos_err=7.5e-10 m  rot_err=  0.00 deg  axis_err=  0.00 deg  OK
…
LR_HR    pos_err=7.5e-10 m  rot_err=  0.00 deg  axis_err=  0.00 deg  OK
LR_FFE   pos_err=1.1e-06 m  rot_err=  0.00 deg  axis_err=  0.00 deg  OK
all joints match
ก่อนแก้: lr_haa … lr_ffe  rot_err = 180.00 deg  BAD
```

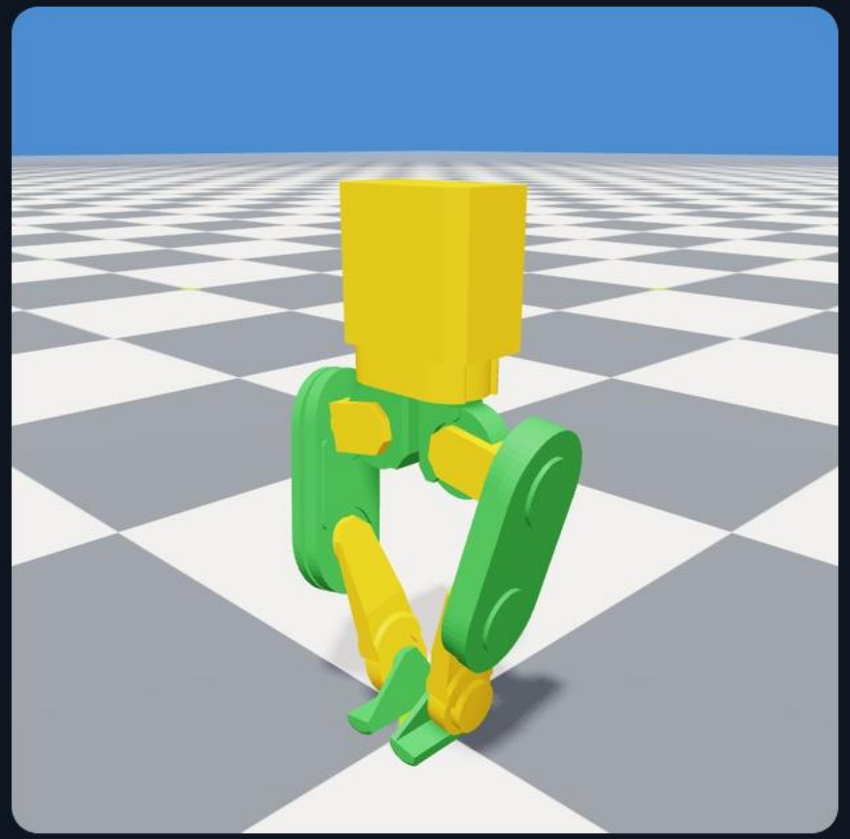
ตรวจงาน: `diff $D/urdf/poclegs.urdf $SUP/expected_output/poclegs.urdf && echo OK`

ตอนทุก joint = 0 ตำแหน่งและทิศของทุก link ต้องตรงกับ USD เดิม ผิดแค่ข้อเดียว policy ที่เทรนมาก็ใช้ไม่ได้แล้ว

HANDS-ON · LAB 6.2 – 6.3

ยืดขา แล้วเทรนใหม่

แก้ URDF แปลงกลับเป็น USD แล้วใส่ใน task เดิม



WHICH NUMBERS?

ยืดต้นขา × s ต้องแก้ 4 ที่ (ต่อขา)

แก้ที่	เดิม	s = 1.3
joint LL_KFE origin	-0.0025 **-0.14** 0	-0.0025 **-0.182** 0
link ll_hfe visual/collision origin	-0.1605 **0.0725** 0	-0.1605 **0.09425** 0
link ll_hfe mesh scale	1 **1** 1	1 **1.3** 1
link ll_hfe inertial origin	y × 1	y × 1.3

mesh วาดในพิกัด CAD เลยต้องคูณ **ทั้ง origin และ scale** ด้วย s ให้ mesh ยืดออกจากข้อสะโพกพอดี

$p' = S \cdot (origin + v) = S \cdot origin + S \cdot v$

หน้าแข้งเหมือนกัน: joint **LL_FFE** origin −0.16 และ link **ll_kfe** ขาขวาก็ทำซ้ำ (LR_*, lr_*)

ASSIGNMENT 6.2 · SCALE THE LEGS

แก้มือก็ได้ หรือใช้ scale_leg.py

```
$ mkdir -p $D/urdf_long && cp -r $D/urdf/meshes $D/urdf_long/
$ uv run python scripts/scale_leg.py $D/urdf/poclegs.urdf \
    $D/urdf_long/poclegs_long.urdf --thigh 1.3 --shank 1.3
wrote …/poclegs_long.urdf
both leg(s) ~0.090 m longer → raise init_state pos z from 0.449 to about 0.539
```

ตัวเลือก

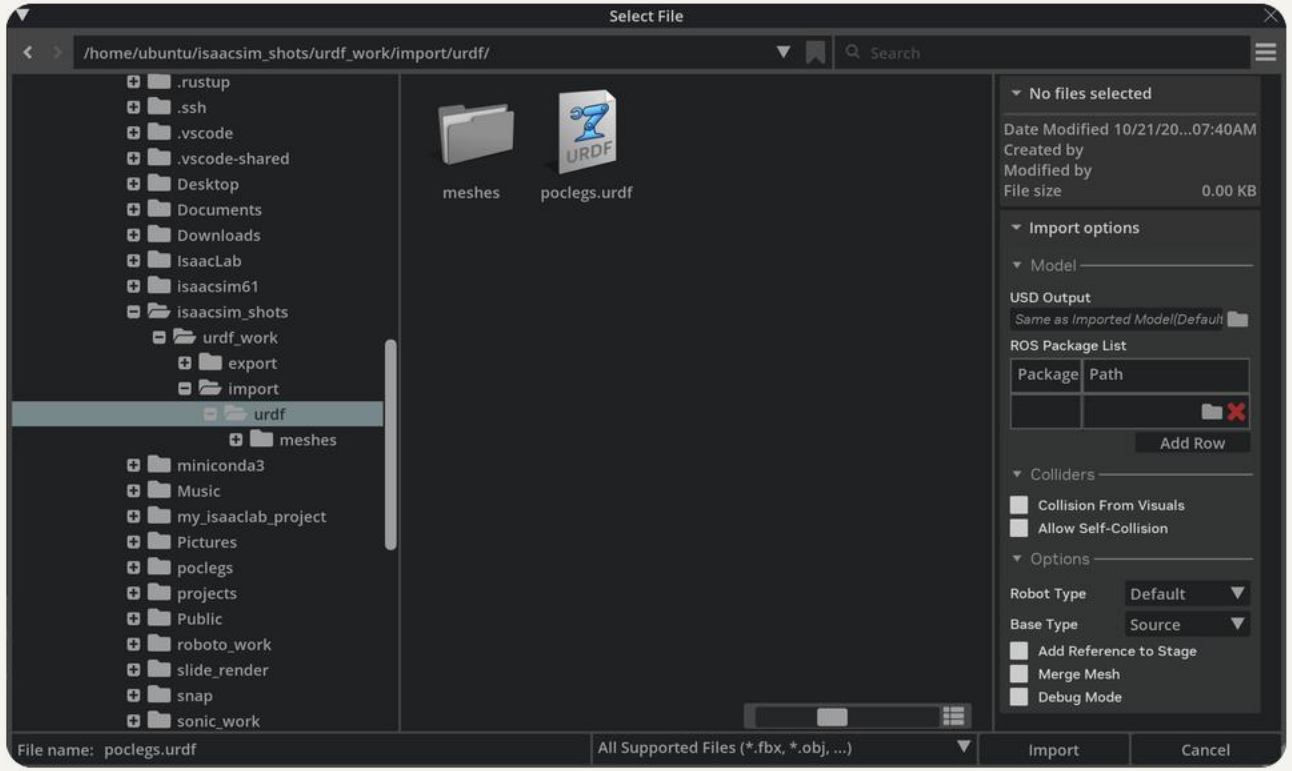
--thigh ต้นขา, **--shank** หน้าแข้ง
--scale_mass มวลเพิ่มตามความยาว
--side left|right แก้ขาข้างเดียว

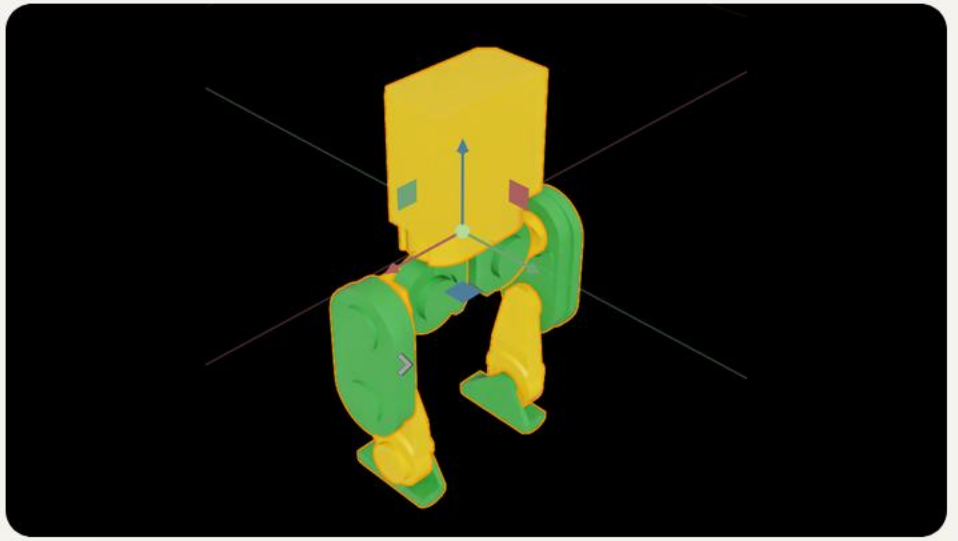
ทำไม z ต้องเพิ่ม

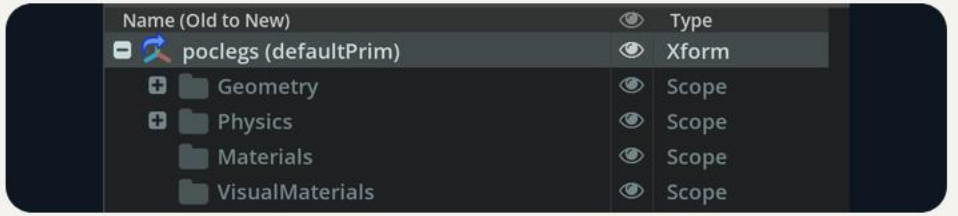
ขายาวขึ้น 0.042 + 0.048 m ถ้าเกิดที่ 0.449 เท้าจะจมพื้นตอนเริ่ม

ทางที่ 1 · ISAAC SIM GUI

URDF → USD: File › Import







1. File › New **2.** File › Import → เลือก `poclegs.urdf` ดู Import options ทางขวา **3.** กด **Import** → ได้ `poclegs/poclegs.usda` + `payloads/` ข้างไฟล์ URDF

ทางที่ 2 · URDF → USD ด้วยสคริปต์ (KIT-LESS)

แปลงกลับเป็น USD โดยไม่เปิด Isaac Sim

```
$ uv run python scripts/urdf_to_usd.py $D/urdf/poclegs.urdf $D/usd_from_urdf  # ×1.0 = ค่าเริ่มของ task
-Flat-Urdf
$ uv run python scripts/urdf_to_usd.py $D/urdf_long/poclegs_long.urdf $D/usd_long
wrote …/usd_long/poclegs_long/poclegs_long.usda
```

```
from isaaclab.sim.converters import (
    UrdfConverter, UrdfConverterCfg)
cfg = UrdfConverterCfg(
    asset_path=urdf, usd_dir=out_dir,
    fix_base=False, merge_fixed_joints=True,
    joint_drive=…PDGainsCfg(stiffness=0.0,
        damping=0.0))
UrdfConverter(cfg)
```

ต้องมี extra importers

```
# pyproject.toml
"isaaclab-dev[importers]"
```

ติดตั้ง isaacsim-asset-isolated (ตัวแปลงที่ไม่ต้องมี Isaac Sim)
ตั้ง gain = 0 เพราะ PD มาจาก actuator ใน Python

PITFALLS: เจอมาแล้วทั้ง 3 ข้อ

USD จาก URDF ไม่เหมือน USD เดิม 100%

1. ชนกันเอง

URDF ไม่มี collision group hull สะโพกเลยทับลำตัว เกิด **base_contact ทุก step**

```
enabled_self_collisions = False
```

2. ซ้อน body

ตัวแปลงวาง link ซ้อนเป็นชั้น Geometry/ base/ll_hr/... contact sensor เลยหา body ไม่เจอ

```
prim_path = "{ENV_REGEX_NS}/
Robot/.*"
```

3. ความสูงตอนเกิด

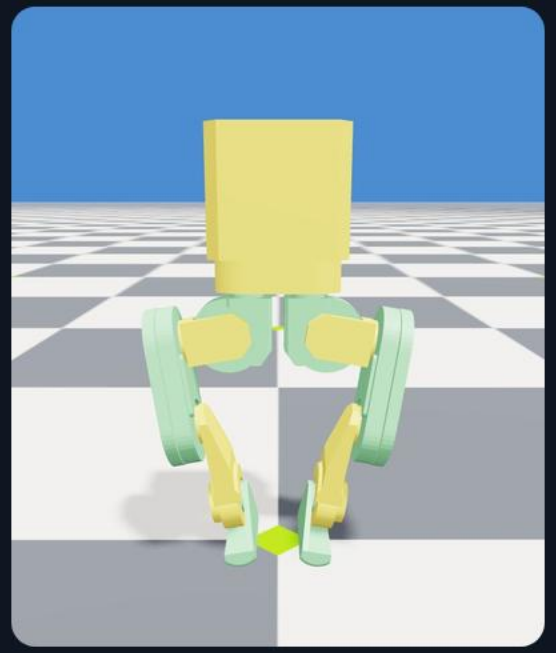
ขายาวขึ้นแต่เกิดที่ z เดิม เท้าจมพื้น หุ่นโดนดีดขึ้นไป 1.1 m แล้วล้ม (วัดมาแล้ว)

```
init_state.pos = [0, 0, 0.539]
```

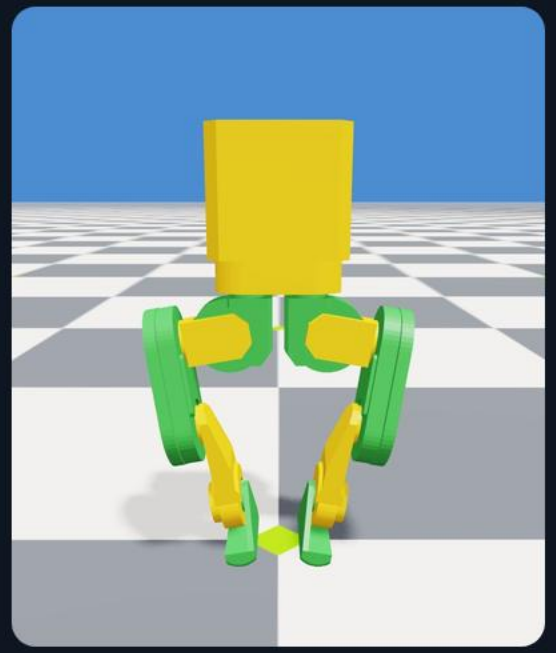
ข้อ 1–2 แก้ไว้แล้วใน task **Poclegs-Velocity-Flat-Urdf** ส่วนข้อ 3 ใส่ตอนสั่งเทรน

CHECK IN THE VIEWER

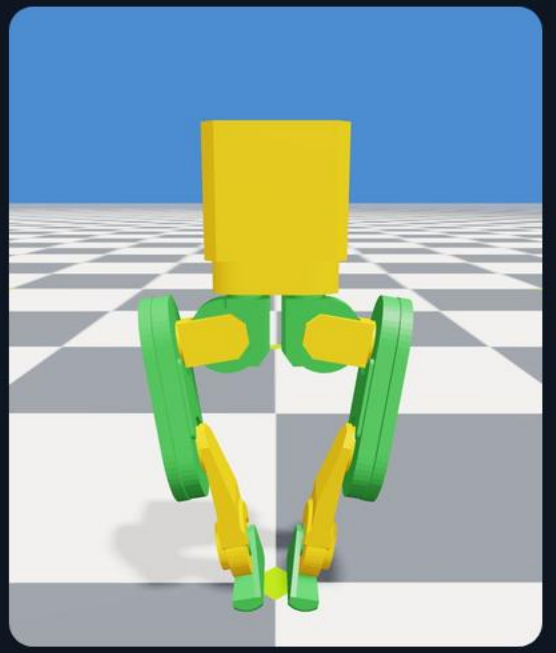
เปิด zero agent ดูให้ชัวร์ก่อนเทรน



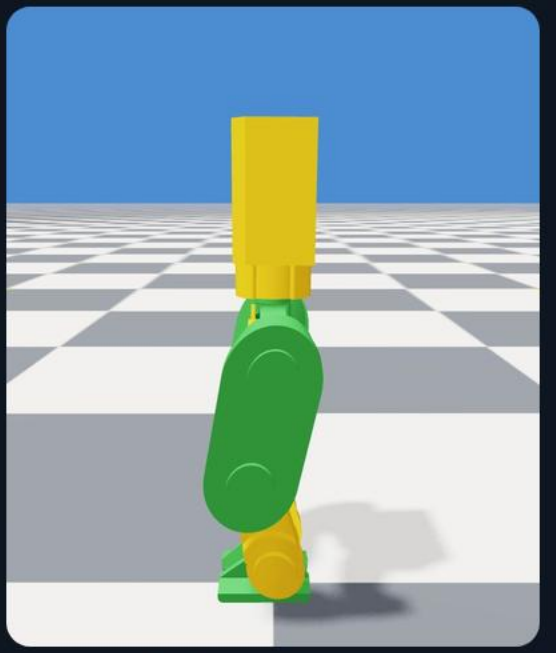
USD เดิม



ผ่าน URDF ×1.0



ขา ×1.3



ขา ×1.3 ด้านข้าง

```
uv run isaaclab zero_agent --task Poclegs-Velocity-Flat-Urdf --viz newton env.scene.robot.spawn.usd_path=$PWD/$D/
usd_long/poclegs_long/poclegs_long.usda "env.scene.robot.init_state.pos=[0,0,0.539]"
```

ASSIGNMENT 6.3 · TRAIN THE NEW BODY

ใช้ task เดิม เปลี่ยนแค่ไฟล์หุ่นกับความสูง

```
$ uv run isaaclab train --rl_library rsl_rl --task Poclegs-Velocity-Flat-Urdf \
    --num_envs 2048 --max_iterations 1500 --run_name long13 \
    env.scene.robot.spawn.usd_path=$PWD/$D/usd_long/poclegs_long/poclegs_long.usda \
    "env.scene.robot.init_state.pos=[0.0,0.0,0.539]"
```

เทียบให้แฟร์

เทียบกับขาเดิมที่ผ่าน URDF (×1.0) ใน task เดียวกัน ไม่ใช่ USD ต้นฉบับที่ inertia คนละแบบ

task ใช้ recipe Tuned

reward แบบ H1, สั่งตามแกน y และปลดเพดานความเร็ว (จาก Week 4)

ค่าที่ override

เก็บไว้ใน logs/.../params/env.yaml ทุกรอบ ถ้าไม่มีเวลาเทรน 1,500 it. ใช้ pretrained/urdf1x/ กับ pretrained/long13/

RESULT: 1,500 IT. × 2,048 ENVS, มอเตอร์เดิม

ขายาว ×1.3 ยืนได้ หมุนได้ แต่ไม่ยอมก้าว

error ความเร็ว xy ระหว่างเทรน [m/s] (ต่ำ = เดินตามคำสั่ง)



สั่ง	×1.0	×1.3
vy +0.4	+0.32 · 3	0.00 · 2
vy −0.4	−0.32 · 3	−0.01 · 8
wz +0.8	+0.74 · 3	+0.85 · 2
ยืนนิ่ง	0 · 1	0 · 2

ความเร็วที่วัดได้ · ครั้งที่ล้ม (64 ตัว × 10 s)

ทั้งคู่ยืนได้ราว 19 s ต่อ episode เท่ากัน ที่ต่างคือการก้าว

FOLLOW-UP: ขายาว + มอเตอร์แข็งขึ้น

kp ×2 และ kd ×2 แล้วขายาวเริ่มก้าวได้

```
… --task Poclegs-Velocity-Flat-Urdf … env.scene.robot.actuators.kfe.stiffness=30.0 (เดิม 15) · haa 30 · hr 20 · ffe 20 · damping ทุกกลุ่ม 3.0 (เดิม 1.5)
# kfe = สะโพกพับ (HFE) + เข่า (KFE) / เปลี่ยน kp กับ kd พร้อมกัน เลยแยกไม่ได้ว่าผลมาจากตัวไหน
```

ขา ×1.3, 600 it.	kp, kd เดิม	kp, kd ×2
vy +0.4	+0.01 · 6	+0.15 · 8
vy −0.4	−0.01 · 3	−0.13 · 16
wz +0.8	+0.68 · 2	+0.01 · 39
ยืนนิ่ง	0 · 1	0 · 3

พอมอเตอร์แข็งพอ policy ก็**กล้าก้าว** แปลว่าปัญหาอยู่ที่ค่ามอเตอร์ ไม่ได้อยู่ที่ RL

แต่ที่ 600 it. ยังหมุนตัวไม่ได้และล้มบ่อย ต้องเทรนนานกว่านี้หรือจูน damping ต่อ

ความเร็ว · ครั้งที่ล้ม (64 ตัว × 10 s)

CHECKPOINT: ก่อนไปต่อ

Lab 6 ผ่านไหม?

6.1: USD → URDF

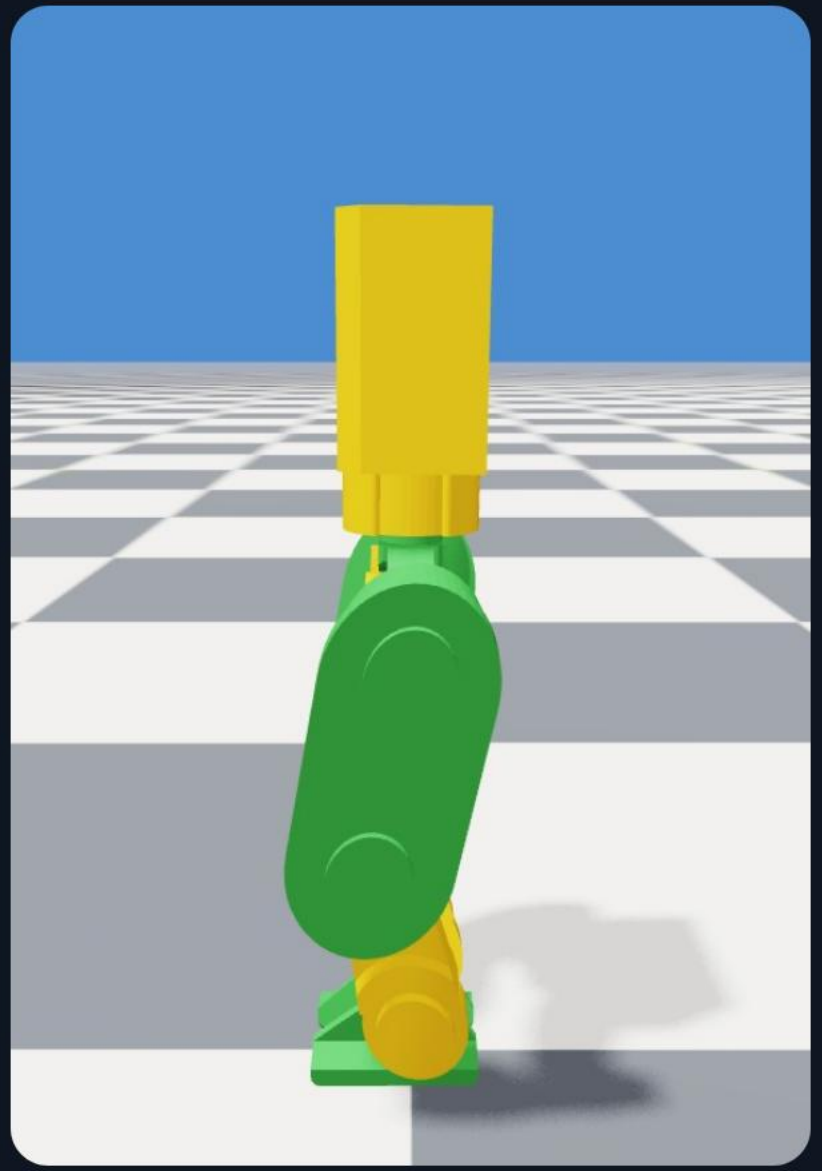
check_urdf.py ขึ้น “all joints match”

6.2: ยืดขา

บอกได้ว่าแก้ link ไหน ยาวเท่าไร และ zero agent เห็นหุ่นขายาวยืนบนพื้น

6.3: เทรนใหม่

บอกได้ว่าเทรนกี่ iteration และผลต่างจากขาเดิมยังไง (วัดด้วย eval_policy.py)



EXERCISES · WEEK 6

ร่างกายเปลี่ยน การเดินเปลี่ยน

มี 4 โจทย์ ใช้ task Poclegs-Velocity-Flat-Urdf เทรน 600 iterations × 2,048 envs (~9 นาที) แล้วเทียบกับขาเดิม ×1.0 ที่ iteration เท่ากัน

EXERCISE 6.1–6.3: ทุกข้อใช้ขั้นตอนเดียวกัน

เปลี่ยนแค่ ชื่อ, ตัวเลือก scale_leg และ z

```
$ D=src/poclegs_lab/assets/data; N=short08  # ชื่อหุ่นใหม่ของข้อนั้น
$ mkdir -p $D/urdf_$N && cp -r $D/urdf/meshes $D/urdf_$N/  # mesh ต้องอยู่ข้าง URDF
$ uv run python scripts/scale_leg.py $D/urdf/poclegs.urdf $D/urdf_$N/poclegs_$N.urdf --thigh 0.8 --shank 0.8
$ uv run python scripts/urdf_to_usd.py $D/urdf_$N/poclegs_$N.urdf $D/usd_$N
$ U=env.scene.robot.spawn.usd_path=$PWD/$D/usd_$N/poclegs_$N/poclegs_$N.usda; Z="env.scene.robot.init_state.pos=[0,0,0.389]"
$ uv run isaaclab train --rl_library rsl_rl --task Poclegs-Velocity-Flat-Urdf --num_envs 2048 \
    --max_iterations 600 --run_name $N $U "$Z"  # ได้ model_599.pt · ~9 นาที
$ EVAL_CMDS="0,0.4,0;0,-0.4,0;0,0,0.8;0,0,0" uv run python scripts/eval_policy.py \
    --task Poclegs-Velocity-Flat-Urdf --num_envs 64 --checkpoint logs/rsl_rl/poclegs_flat/<run>_$N/model_599.pt $U "$Z"
```

ตัวเทียบ (แจกให้แล้ว)

`pretrained/urdf1x/model_600.pt`

ขา ×1.0: eval ด้วย task เดียวกัน **ไม่ต้อง**ใส่ $U "$Z"

`pretrained/long13/model_600.pt`

ขา ×1.3: ใส่ usd_long + z 0.539 (จาก Lab 6.2)

ค่าของแต่ละข้อ

6.1 N=short08, --thigh 0.8 --shank 0.8, z 0.389

6.2 N=longmass, --thigh 1.3 --shank 1.3 --scale_mass, z 0.539

6.3 N=asym, --shank 1.15 --side left, z 0.473

ตอน eval ต้องใส่ $U "$Z" ให้ตรงกับตอนเทรน ไม่งั้นจะวัด policy บนหุ่นตัวเดิม บน RTX 3060 6 GB เทรน 600 it. ราว 8–9 นาที/ข้อ

EXERCISE 6.1 · SHORTER LEGS

ขาสั้นลง 20% เดินดีขึ้นหรือแย่ลง?

6.1

```
$ mkdir -p $D/urdf_short08 && cp -r $D/urdf/meshes $D/urdf_short08/
$ uv run python scripts/scale_leg.py $D/urdf/poclegs.urdf $D/urdf_short08/poclegs_short08.urdf --thigh 0.8 --shank 0.8
$ uv run python scripts/urdf_to_usd.py $D/urdf_short08/poclegs_short08.urdf $D/usd_short08
$ uv run isaaclab train --rl_library rsl_rl --task Poclegs-Velocity-Flat-Urdf --max_iterations 600 \
    env.scene.robot.spawn.usd_path=$PWD/$D/usd_short08/poclegs_short08/poclegs_short08.usda \
    "env.scene.robot.init_state.pos=[0,0,0.389]"
# ต้อง override 2 ค่าเสมอ: usd_path (ไฟล์หุ่นใหม่ ถ้าลืมจะเทรนหุ่นเดิม) + pos = [x, y, z] ของลำตัวตอนเกิด (m)
# z = 0.449 - 0.2 × (ต้นขา 0.14 + หน้าแข้ง 0.16) = 0.389 ใส่ " " ครอบเพราะมี [ ]
```

ทายก่อนเทรน

ขาสั้น จุดศูนย์ถ่วงต่ำ ทรงตัวง่าย แต่ก้าวสั้น ต้องก้าวถี่ขึ้นถึงจะเร็วเท่าเดิม

โจทย์

1. เทรน 600 iterations แล้ว eval ด้วยคำสั่ง vy ±0.4
2. เทียบกับขา ×1.0: pretrained/urdf1x/model_600.pt
3. อะไรดีขึ้น อะไรแย่ลง?

เฉลย EXERCISE 6.1

ขาสั้นไม่ได้ง่ายกว่า ก้าวสั้นจนแทบไม่ไปไหน

1. ที่ 600 iterations ขา ×0.8 เดินหน้าได้ 0.06 m/s (×1.0 ได้ 0.22) และแทบหมุนตัวไม่ได้ (0.08 vs 0.27 rad/s)
2. ระหว่างเทรนก็ล้มบ่อยกว่า: 24% ของ episode จบเพราะล้ม (×1.0: 17%) และยืนได้ 860 vs 936 step
3. reward ยังตั้งไว้สำหรับขาเดิม (air time 0.4 s, ช่วงคำสั่ง ±0.5 m/s) แต่ขาสั้นต้องก้าวถี่กว่าเดิมถึงจะเร็วเท่ากัน

สั่ง	×1.0	×0.8
vy +0.4	+0.22 · 18	+0.06 · 12
vy −0.4	−0.04 · 27	0.00 · 5
wz +0.8	+0.27 · 23	+0.08 · 31
ยืนนิ่ง	0 · 11	0 · 4

ความเร็ว · ครั้งที่ล้ม (64 ตัว × 10 s) ใช้ model_600 ทั้งคู่ เทรนรอบเดียว (seed เดียว)

เปลี่ยนสัดส่วนหุ่นเมื่อไร ค่า reward / คำสั่ง / ความสูงที่จูนไว้กับหุ่นเดิมก็อาจไม่เหมาะแล้ว ต้องจูนใหม่ให้เข้ากับร่างกาย

EXERCISE 6.2 · MASS FOLLOWS LENGTH

ขายาวขึ้น ของจริงก็ต้องหนักขึ้น

6.2

```
$ mkdir -p $D/urdf_longmass && cp -r $D/urdf/meshes $D/urdf_longmass/
$ uv run python scripts/scale_leg.py $D/urdf/poclegs.urdf $D/urdf_longmass/poclegs_longmass.urdf \
    --thigh 1.3 --shank 1.3 --scale_mass
# ต้นขา 3.0 → 3.9 kg · หน้าแข้ง 1.0 → 1.3 kg (ต่อข้าง) · รวม 17.5 → 19.9 kg
```

ระวัง

scale_mass เปลี่ยนแค่ **mass** ส่วน inertia ใน URDF ยังเป็นค่าเดิม (ของจริงต้องโตตามความยาว²)

โจทย์

1. เทรน 600 iterations ที่ z = 0.539
2. เทียบกับขา ×1.3 มวลเดิม: pretrained/long13/model_600.pt
3. มอเตอร์เดิมยังพอไหม?

เฉลย EXERCISE 6.2

หนักขึ้น 2.4 kg แต่มอเตอร์เท่าเดิม ยังไม่ก้าว แถมหมุนได้น้อยลง

1. ที่ 600 iterations ขา ×1.3 ทั้งแบบมวลเดิมและมวลเพิ่ม **ยืนได้แต่ไม่เดิน** (vy ≈ 0)
2. มวลเพิ่มทำให้หมุนตัวได้น้อยลง (0.50 vs 0.68 rad/s) และ error ระหว่างเทรนสูงกว่า (0.36 vs 0.32 m/s)
3. ขายาวและหนัก สะโพกต้องใช้แรงบิดมากขึ้นทั้งจากแขนโมเมนต์และน้ำหนัก แต่ kp / แรงสูงสุดยังเท่าเดิม มอเตอร์เดิมเลย**ไม่พอ**

สั่ง	×1.3	×1.3 + มวล
vy +0.4	+0.01 · 6	0.00 · 1
vy −0.4	−0.01 · 3	0.00 · 5
wz +0.8	+0.68 · 2	+0.50 · 1
ยืนนิ่ง	0 · 1	0 · 7

ความเร็ว · ครั้งที่ล้ม (64 ตัว × 10 s) model_600 ทั้งคู่

ออกแบบหุ่นใหม่ ต้องคิดเรื่องมอเตอร์ไปด้วย ยืดขาแต่ไม่อัปเกรดมอเตอร์ RL ก็ช่วยไม่ได้ (ดูการทดลองเพิ่ม kp ในหน้า “ขายาว + มอเตอร์แข็งขึ้น”)

EXERCISE 6.3 · ASYMMETRIC LEGS

ขาซ้ายยาวกว่าขวา 2.4 cm

6.3

```
$ mkdir -p $D/urdf_asym && cp -r $D/urdf/meshes $D/urdf_asym/
$ uv run python scripts/scale_leg.py $D/urdf/poclegs.urdf $D/urdf_asym/poclegs_asym.urdf \
    --shank 1.15 --side left
left leg(s) ~0.024 m longer → raise init_state pos z from 0.449 to about 0.473
```

ทำไมน่าสนใจ

หุ่นจริงประกอบไม่เท่ากันเป๊ะ (ชิ้นงาน 3D print, ข้อต่อหลวม) policy จะชดเชยเองได้ไหม?

โจทย์

1. เทรน 600 iterations ที่ z = 0.473
2. สั่งเดินตรง (vy) แล้วดู vx กับ wz หุ่นเดินเบี้ยวไหม?
3. ถ้าเอา policy ขาเท่ากัน (pretrained/urdf1x/model_1499.pt) มาใช้กับหุ่นนี้ล่ะ?

เฉลย EXERCISE 6.3

ขาต่างกัน 2.4 cm ก็พอให้หุ่นเดินเบี้ยว

1. เทรนกับหุ่นขาไม่เท่าเอง 600 it. แทบไม่เดินเลย (vy +0.08 / 0.00) แย่กว่าขาเท่ากันที่ iteration เท่ากัน (+0.22 / −0.04)
2. เอา policy ขาเท่ากัน (1,500 it.) มาใช้: ยังเดินได้ แต่**ไถลข้าง** vx −0.085 m/s ตอนเดินหน้า และ**หมุน** −0.16 rad/s ตอนเดินถอย
3. ตอนสั่งหมุนตัวล้มไป 24 ครั้ง (บนหุ่นที่เทรนมา: 3) เพราะ policy ไม่เคยเจอหุ่นที่ไม่สมมาตร

policy ขาเท่า	หุ่นขาเท่า	หุ่นขาไม่เท่า
vy +0.4 → vx	−0.02	−0.09
vy −0.4 → wz	−0.03	−0.16
wz +0.8 (ล้ม)	3	24
ยืนนิ่ง → vy	−0.01	−0.04

urdf1x model_1499, 64 ตัว × 10 s, ค่าที่ควรเป็น 0

หุ่นจริงไม่สมมาตรเป๊ะ ทางแก้คือเทรนแบบสุ่มความยาว/มวล (domain randomization) หรือวัดหุ่นจริงแล้วใส่ใน URDF ให้ตรง

EXERCISE 6.4 · SPAWN HEIGHT

ลืมยกความสูงตอนเกิด จะเป็นอะไร?

6.4

```
$ uv run isaaclab zero_agent --task Poclegs-Velocity-Flat-Urdf --num_envs 8 --viz newton \
    env.scene.robot.spawn.usd_path=$PWD/$D/usd_long/poclegs_long/poclegs_long.usda
# ไม่ใส่ init_state.pos → เกิดที่ z = 0.449 m (ความสูงของขาเดิม ×1.0)
```

โจทย์

1. ในวินาทีแรก หุ่นในหน้าต่างทำอะไร?
2. ทำไม? (ขายาวขึ้น 9 cm แต่เกิดที่เดิม)
3. ถ้าเทรนแบบนี้ ทุกครั้งที่ reset จะเกิดอะไร?

ค่าที่ถูก

z = 0.449 + ความยาวที่เพิ่ม
scale_leg.py พิมพ์ค่าให้: **0.539**

```
"env.scene.robot.init_state.pos=[0,0,0.539]"
```

เฉลย EXERCISE 6.4

เท้าจมพื้น 9 cm จน solver ดีดหุ่นลอยขึ้นฟ้า

1. ลำตัวเกิดที่ 0.45 m โดนดีดเป็น 0.52 m ตั้งแต่ step แรก พุ่งขึ้นไป **1.1 m** ภายใน 0.4 s แล้วตีลังกาลงมาเจอ base_contact ที่ step 38
2. ขายาวขึ้น 0.09 m แต่ลำตัวเกิดที่เดิม เท้าเลยอยู่ใต้พื้น contact solver ก็ดันออกด้วยแรงมหาศาลในไม่กี่ step
3. ทุกครั้งที่ reset หุ่นจะโดนดีด policy ต้องเรียนจาก episode ที่พังตั้งแต่เริ่ม เลย แทบเทรนไม่ได้

step	เวลา	z ลำตัว
0	0.00 s	0.52 m
10	0.20 s	1.04 m
20	0.40 s	1.14 m
30	0.60 s	0.83 m
38	0.76 s	0.30 m (ล้ม)

ขา ×1.3 เกิดที่ z 0.449, zero action, หุ่นตัวแรก (ตัวอื่นใกล้เคียงกัน)

เปลี่ยนร่างกายหุ่นเมื่อไร ต้องแก้ init_state (ความสูง + ท่าข้อต่อ) ตามทุกครั้ง แล้วเปิด zero agent ดูก่อนเทรน